

Web & Web Proxy Server

Protocol:

If request HTTP version is NOT HTTP 1.1, return 505 HTTP Version Not Supported. Then, if HTTP request method is GET, then check for valid URL. If URL is invalid return 404 Not Found. If GET request "If-Modified-Since" date is after September 20th, 2026 then return 304 Not Modified. If URL is valid and file does not require authentication, return 200 OK and file content. If file is authenticated, and password is valid but does not have correct permissions to access file then return 403 Forbidden. If valid password has permission then return 200 OK and file content. If password is invalid for authenticated file then return 404 Not Found.

Status Codes:

200 OK

Request Requirements: Valid request URL and correct authentication if required.

Test HTTP Request Message(s):

GET /test.html HTTP/1.1

Host: localhost:8080

User-Agent: curl/8.21.0

Accept: */*

GET /adminonly.html HTTP/1.1

Host: localhost:8080

User-Agent: curl/8.21.0

Accept: */*

Authentication: Bearer admin123

304 Not Modified

Request Requirements: If GET request If-modified-since date is before object-not-modified-before date.

Test HTTP Request Message(s):

GET /test.html HTTP/1.1

Host: localhost:8080

User-Agent: curl/8.21.0

Accept: */*

If-Modified-Since: Tue, 21 Nov 2050 08:00:00 GMT

403 Forbidden

Request Requirements: Valid request authentication details but insufficient permission.

Test HTTP Request Message(s):

GET /adminonly.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.21.0
Accept: */*
Authentication: Bearer student321

404 Not Found

Request Requirements: Invalid GET request URL or incorrect password in request authentication.

Test HTTP Request Message(s):

GET /notfound.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.21.0
Accept: */*

GET /adminonly.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.21.0
Accept: */*
Authentication: Bearer randomkid

505 HTTP Version Not Supported

Request Requirements: If specified HTTP version in request message is outdated.

Test HTTP Request Message(s):

GET /test.html HTTP/1.0
Host: localhost:8080
User-Agent: curl/8.21.0
Accept: */*

Testing:

200 OK

```

C:\Users\gabec\Repos\371-fp>curl --no-keepalive -v http://localhost:8080/test.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
>
< HTTP/1.1 200 OK
* no chunk, no close, no size. Assume close to signal end
<
Date: Mon, 10 Aug 2026 22:28:28 GMT
Expires: Sun, 20 Sep 2026 00:00:00 GMT
Transfer-Encoding: chunked
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1">

</head>

<body>

  <p>Congratulations! Your Web Server is Working!</p>

</body>

</html>
* abort upload
* shutting down connection #0

```

304 Not Modified

```

C:\Users\gabec\Repos\371-fp>curl --header "If-Modified-Since: Tue, 21 Nov 2050 08:00:00 GMT" -v http://localhost:8080/test.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
> If-Modified-Since: Tue, 21 Nov 2050 08:00:00 GMT
>
* Request completely sent off
< HTTP/1.1 304 Not Modified
< Date: Mon, 10 Aug 2026 22:30:55 GMT
< Expires: Sun, 20 Sep 2026 00:00:00 GMT
< Transfer-Encoding: chunked
* Connection #0 to host localhost left intact

```

505 HTTP Not Supported

```
C:\Users\gabec\Repos\371-fp>curl --no-keepalive -v --http1.0 http://localhost:8080/test.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /test.html HTTP/1.0
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
>
< HTTP/1.1 505 HTTP Version Not Supported
* no chunk, no close, no size. Assume close to signal end
<
* abort upload
* shutting down connection #0
```

404 Not Found

```
C:\Users\gabec\Repos\371-fp>curl --no-keepalive -v http://localhost:8080/notfound.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /notfound.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
* no chunk, no close, no size. Assume close to signal end
<
* abort upload
* shutting down connection #0
```

200 OK

```

C:\Users\gabec\Repos\371-fp>curl --header "Authentication: Bearer admin123" -v http://localhost:8080/adminonly.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /adminonly.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
> Authentication: Bearer admin123
>
< HTTP/1.1 200 OK
* no chunk, no close, no size. Assume close to signal end
<
Date: Mon, 10 Aug 2026 22:36:24 GMT
Expires: Sun, 20 Sep 2026 00:00:00 GMT
Transfer-Encoding: chunked
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1">

</head>

<body>

  <p>Successful admin authentication!</p>

</body>

</html>
* abort upload
* shutting down connection #0

```

403 Forbidden

```

C:\Users\gabec\Repos\371-fp>curl --header "Authentication: Bearer student321" -v http://localhost:8080/adminonly.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /adminonly.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
> Authentication: Bearer student321
>
< HTTP/1.1 403 Forbidden
* no chunk, no close, no size. Assume close to signal end
<
Date: Mon, 10 Aug 2026 22:37:09 GMT
* abort upload
* shutting down connection #0

```

404 Not Found

```
C:\Users\gabec\Repos\371-fp>curl --header "Authentication: Bearer randomkid" -v http://localhost:8080/adminonly.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /adminonly.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.15.0
> Accept: */*
> Authentication: Bearer randomkid
>
< HTTP/1.1 404 Not Found
* no chunk, no close, no size. Assume close to signal end
<
* abort upload
* shutting down connection #0
```

Overall Calls to Web Server:

```
PS C:\Users\gabec\repos\371-fp> py main.py
Listening on port 8080 ...
GET /test.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*

GET /test.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*
If-Modified-Since: Tue, 21 Nov 2050 08:00:00 GMT

GET /test.html HTTP/1.0
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*

GET /notfound.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*

GET /adminonly.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*
Authentication: Bearer admin123

GET /adminonly.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*
Authentication: Bearer student321

GET /adminonly.html HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.15.0
Accept: */*
Authentication: Bearer randomkid
```

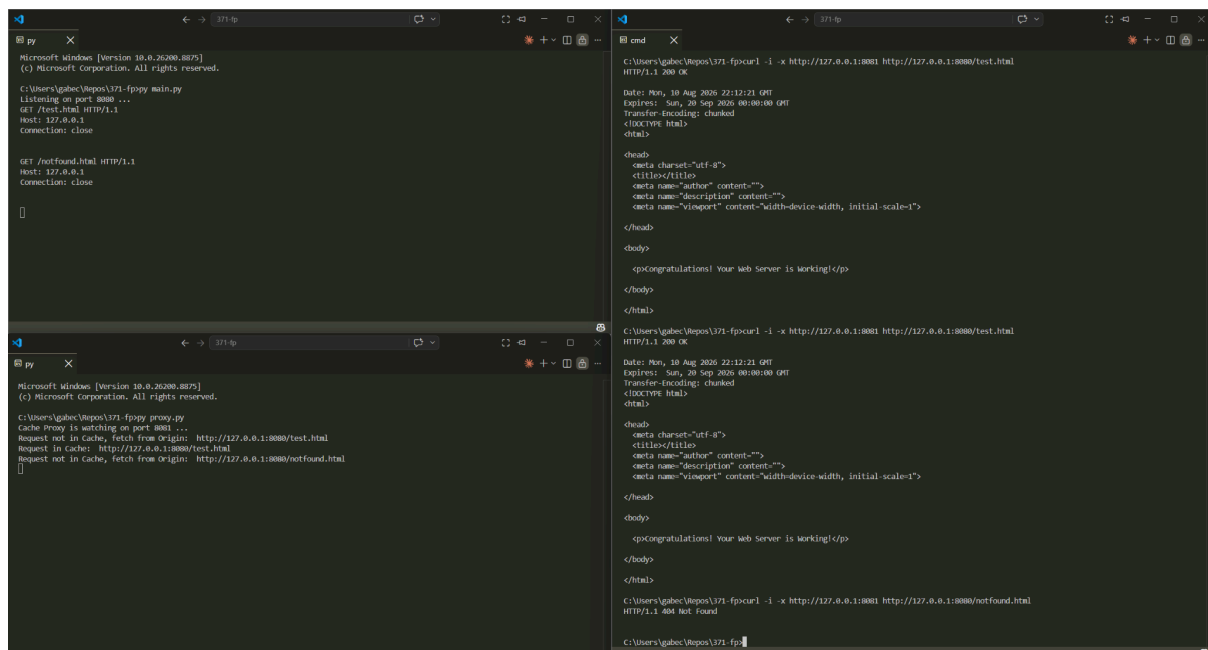
Performance:

Proxy:

A web server that hosts my files would serve them directly to me, very straightforward. A proxy server does not have that same luxury, it must first process the incoming client request and then find if that Request's Object is in its cache (if it is, the proxy serves it back to the client). If the Object is not in the cache, it must find the relevant Origin Server that has the Object, request for it, then return the Object to the client.

To create a minimal proxy server with Web caching, it must route all Client HTTP requests through itself rather than directly to the Origin Servers. When the Proxy receives a Client Request, it needs to handle the request by checking the headers and data, as it doesn't have a local file/resource on hand. Then, it should check for the Request's Object in it's Cache and return it to the client, if found. If not found, it should contact its Origin Server and send a request for the Object, becoming a Client itself. Once the Origin Server sends a Response, the Proxy should cache a copy of the Response in itself and return the original Response back to the Client. This Proxy will be using GET requests.

Testing:



```
Microsoft Windows [Version 10.0.20308.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gabec\Repos\371-fpppy main.py
Listening on port 8080 ...
GET /test.html HTTP/1.1
Host: 127.0.0.1
Connection: close

GET /notfound.html HTTP/1.1
Host: 127.0.0.1
Connection: close

[]

C:\Users\gabec\Repos\371-fpppy proxy.py
Cache Proxy is watching on port 8081 ...
Request not in Cache, fetch from Origin: http://127.0.0.1:8080/test.html
Request in Cache: http://127.0.0.1:8080/test.html
Request not in Cache, fetch from Origin: http://127.0.0.1:8080/notfound.html

[]

C:\Users\gabec\Repos\371-fpppy curl -i -x http://127.0.0.1:8081 http://127.0.0.1:8080/test.html
HTTP/1.1 200 OK

Date: Mon, 10 Aug 2025 22:12:21 GMT
Expires: Sun, 20 Sep 2026 00:00:00 GMT
Transfer-Encoding: chunked
<DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>
  <p>Congratulations! your web server is working</p>
</body>
</html>

C:\Users\gabec\Repos\371-fpppy curl -i -x http://127.0.0.1:8081 http://127.0.0.1:8080/test.html
HTTP/1.1 200 OK

Date: Mon, 10 Aug 2025 22:12:21 GMT
Expires: Sun, 20 Sep 2026 00:00:00 GMT
Transfer-Encoding: chunked
<DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>
  <p>Congratulations! your web server is working</p>
</body>
</html>

C:\Users\gabec\Repos\371-fpppy curl -i -x http://127.0.0.1:8081 http://127.0.0.1:8080/notfound.html
HTTP/1.1 404 Not Found
```

The test procedure I chose were 3 curl requests.

1. `curl -i -x http://127.0.0.1:8081 http://127.0.0.1:8080/test.html`
2. Duplicate run of first curl command
3. `curl -i -x http://127.0.0.1:8081 http://127.0.0.1:8080/notfound.html`

The top-left terminal is the main web server running on `main.py`, bottom-left terminal is the proxy server running on `proxy.py`, and the right terminal showcases the three GET requests used to test. Inside the `proxy.py` If-Else branch, I included a print

statement to better outline when the Proxy's Cache is called. Requests were routed through the proxy using curl's -x flag, which directs curl to send the request to the proxy while specifying the actual destination as an absolute-form URI (e.g., curl -x http://127.0.0.1:8081 <http://127.0.0.1:8080/test.html>)

In the first request, since the proxy has no stored response, it parses for the destination and opens a connection to the origin server, retrieves the page, stores a copy in its cache, and returns it to the client.

In the second request, the same command is run and the proxy is able to find the URL in its cache already, returning the stored response directly to the client.

In the third request, since this path doesn't exist in the origin, it returns whatever status code the origin web server returns, which is 404.

Why a Multi-Threaded Web Server?

Originally, the design used a single while True loop that called accept(), then processed the req/resp cycle before accepting a new connection. Meaning, a client would need to wait for the first client's connection to close, before their request is handled. We moved all that handler logic into a function aptly called clientHandler and changed that loop to create a threading.Thread instance every time a connection is made that returns the accept() rather than blocking the current client. By making the server multi-threaded, each client connection will be handled on its own thread independently from other connections, which is way faster than needing to wait in line before the server finishes the client request ahead of you.

Preventing Head of Line Blocking:

Our existing reqPaths do not display the HOL blocking scenarios in Slide 37/38 since they only ever transmit one object over the client connection (simple HTML pages). The scenario shown in the slides requires multiple objects sharing a singular connection, with large object's frames being processed alongside the smaller object's frames concurrently, but our connections close once a singular request is served so there is no queue.